

Auditoria KAVANA RouteFleet

Hermes Agent

2026-07-13

Índice

AUDITORIA TECNICA PROFESIONAL — KAVANA ROUTEFLEET	1
1. RESUMEN EJECUTIVO	1
2. STACK TECNOLÓGICO	2
3. ARQUITECTURA	2
4. DOCUMENTACION DEL PROYECTO (README.md)	2
5. DECISIONES ESTRATEGICAS	3
2026-04-29: Optimizacion Operativa y Rediseño Táctico	3
2026-05-01: Unificación de Marca y Conectividad Ubicua	3
6. REGISTRO DE TAREAS (task.md)	3
7. ANEXO TECNICO DE AUDITORIA	3
7.1 Endpoints de la API (server/src/routes/api.js)	3
7.2 Cobertura de tests (19/19 verdes)	4
7.3 Verificación de arranque (real, no asumido)	4
8. DEUDA TECNICA Y OBSERVACIONES (para consultoria)	4
9. COMO VISITARLO COMO CONSULTORIA	4

AUDITORIA TECNICA PROFESIONAL — KAVANA ROUTEFLEET

Proyecto: KAVANA RouteFleet (anteriormente KAVANA Logistics) **Division:** Kavana Systems — Logística de campo para sector metalúrgico **Fecha:** 2026-07-13 **Autor del informe:** Hermes Agent (perfil jobhunter) **Estado:** v1.0 — Backend funcional, 19 tests automatizados, CI activa **Repositorio:** <https://github.com/kavanasystemsinfo-ui/kavana-RouteFleet>

1. RESUMEN EJECUTIVO

KAVANA RouteFleet es una aplicación de gestión de logística de campo diseñada para fabricantes del sector metalúrgico (estanterías, puntales, bandejas). Resuelve un problema industrial real: el reparto de materiales desde fábrica hasta obra, con control de costes OPEX, OCR de albaranes y prueba de entrega (POD) digital.

Este informe unifica toda la documentación del repositorio GitHub y añade un análisis técnico de auditoría orientado a revisión por consultoría IT o Tech Lead.

Puntos destacables tras la profesionalizacion de 2026-07-13: - El backend estaba incompleto (servicios referenciados pero ausentes). Se reconstruyo con TDD: 19 tests verdes, servidor arranca y todos los endpoints responden. - Se cerro la funcionalidad nucleo de logistica: POD en PDF con firma + geolocalizacion. - Se anadio fallback de rutas: si la IA (DeepSeek/OpenRouter) cae, un algoritmo greedy local garantiza el reparto. El sistema nunca se detiene. - CI en GitHub Actions corre tests + build en cada push. - Marca unificada: KAVANA Logistics -> KAVANA ROUTEFLEET.

2. STACK TECNOLÓGICO

Capa	Tecnologia	Notas
Frontend	React 18 + Vite	Estilos inline (estabilidad visual)
UI	Lucide-React, Framer Motion	Electric Orange #FF3D00
Backend	Node.js + Express	Servicios modulares testeables
Datos	SQLite (better-sqlite3)	Local, sin servidor externo
OCR	Tesseract.js	Opcional, degrada a entrada manual
PDF	pdfkit	Generacion de POD
IA Rutas	DeepSeek v3 via OpenRouter	Con fallback greedy local
Tests	node -test (nativo)	19 tests, sin frameworks externos
CI	GitHub Actions	Tests backend + build cliente

3. ARQUITECTURA

client/ (React, Vite) App.jsx AdminDashboard.jsx (Torre Control) Scanner.jsx (Kavana Lens OCR) SignaturePad.jsx (POD) IncidentModal.jsx	server/ (Express API) src/ index.js (arranque + inyeccion DB) db.js (SQLite + queries) routes/api.js (endpoints REST) services/ addressCleaner.js (limpieza OCR) ocrService.js (OCR + fallback) aiService.js (IA + greedy) routeOptimizer.js (greedy puro) pdfService.js (POD PDF) tests/ (19 tests node:test)
---	--

La separacion de la logica pura (routeOptimizer, addressCleaner) de las dependencias de IO (fetch, pdfkit, db) permite testear el comportamiento sin mocks complejos.

4. DOCUMENTACION DEL PROYECTO (README.md)

KAVANA ROUTEFLEET es la division logistica de Kavana Systems disenada para fabricantes del sector metalurgico. Sistema de gestion de campo tactico que prioriza eficiencia, control de costes y trazabilidad ISO 9001.

Innovaciones clave: - Identidad Electric Orange (#FF3D00) para visibilidad en tablets industriales. - Arquitectura ubica: Torre de Control (desktop) + App Operario (movil) via red local. - IA Routing Engine:

DeepSeek v3 via OpenRouter, con fallback greedy local. - Kavana Lens: OCR de albaranes industriales con filtrado de materiales. - POD digital: PDF de entrega con firma y geolocalizacion.

Calidad y testing (TDD): el backend se construyo con tests antes del codigo. Ejecutar: `cd server && npm install && npm test`. La CI corre estos tests y el build del cliente en cada push a main.

5. DECISIONES ESTRATEGICAS

2026-04-29: Optimizacion Operativa y Redisenio Tactical

1. Motor de IA (DeepSeek v3): integracion via OpenRouter para optimizacion de rutas. Coste ~0 y conocimiento semantico de geografia espanola, sin geocodificacion lat/lng.
2. UI con estilos inline: migracion de clases externas a inline en React por fallos de compilacion con Tailwind en local. Estetica "Premium Tactical" estable sin dependencias.
3. OCR con filtrado logico: post-procesamiento que limpia simbolos de tabla y filtra palabras clave industriales (Puntales, Largueros). Direccion limpia para GPS al 95%.
4. Mapa en vivo "Zero-Cost": iframe de Google Maps con filtros CSS (invert/hue-rotate) para simular Dark Mode sin tarjeta de credito ni API de paga.

2026-05-01: Unificacion de Marca y Conectividad Ubicua

5. Identidad "Electric Orange" (#FF3D00): contraste para entornos industriales con luz.
 6. Enrutamiento dinamico de red: `window.location.hostname` en lugar de localhost, para uso en tablets/moviles en la misma WiFi sin deploy en la nube.
 7. Modelo de costes OPEX real-time: Sueldo Operario/Hora + Desgaste Vehiculo/Km, para ver el impacto del trafico y la eficiencia en el margen de beneficio por entrega.
-

6. REGISTRO DE TAREAS (task.md)

Completado 2026-04-29: OCR filtrado, limpieza de direcciones, OpenRouter, redisenio UI, zoom mapa, migracion a estilos inline.

Completado 2026-05-01: Electric Orange, multi-dispositivo, gestion de paradas, firewall, modelo OPEX.

Completado 2026-07-13 (profesionalizacion TDD + CI): - Backend reconstruido y completado (db/ocr/ai/pdf services). Servidor arranca. - POD en PDF con firma y geolocalizacion (funcionalidad nucleo cerrada). - Fallback de rutas (greedy local si OpenRouter cae). - 19 tests automatizados con node -test. - CI en GitHub Actions (tests + build). - Unificacion de marca Logistics -> ROUTEFLEET.

Proximos pasos: "Hambre de Material" (consumo real vs teorico), multi-tenant, historico de rutas por operario, notificaciones push, tests del cliente.

7. ANEXO TECNICO DE AUDITORIA

7.1 Endpoints de la API (server/src/routes/api.js)

Metodo	Ruta	Funcion
GET	/api/dashboard-data	Metricas Torre de Control + PODs + incidencias
GET	/api/settings	Costes OPEX (cost_per_km, cost_per_hour)
PUT	/api/settings	Actualiza costes OPEX
GET	/api/stops	Lista paradas
POST	/api/ocr	OCR de albaran (multipart image)
POST	/api/ocr_manual	Alta manual de parada
POST	/api/optimize	Optimizacion de ruta (IA + fallback)
PATCH	/api/stops/:id	Actualiza parada / genera POD al entregar
DELETE	/api/stops/:id	Borra parada
DELETE	/api/stops	Limpia ruta
POST	/api/stops/:id/incident	Registra incidencia

7.2 Cobertura de tests (19/19 verdes)

- addressCleaner: limpieza de simbolos y materiales industriales.
- routeOptimizer: orden greedy deterministico (empieza en origen, visita todas).
- aiService: fallback a greedy cuando no hay API key.
- db: altas/bajas/consultas SQLite en :memory:.
- pdfService: genera PDF valido (%PDF-) con firma y geo.
- ocrService: procesa imagen sin fallar si Tesseract ausente.

7.3 Verificacion de arranque (real, no asumido)

Servidor iniciado en puerto 5099 de prueba: - GET /api/settings -> {"cost_per_km":0.3,"cost_per_hour":15}
- POST /api/optimize -> engine "local-greedy", ruta [1,2] - GET /api/dashboard-data -> metricas correctas

8. DEUDA TECNICA Y OBSERVACIONES (para consultoria)

Fortalezas que un Tech Lead valora: 1. Product mindset: soluciones pragmaticas bajo restricciones (iframe CSS, estilos inline). 2. Mentalidad de negocio: modelo OPEX real, no solo codigo. 3. Uso eficiente de IA: DeepSeek por conocimiento semantico a coste minimo. 4. Resiliencia: fallback greedy -> el reparto nunca se detiene. 5. TDD + CI: 19 tests, pipeline automatico. Disciplina senior autodidacta.

Puntos a madurar (con propuesta concreta): 1. Multi-tenant: SQLite actual es mono-tenant. Proximo paso: aislamiento por cliente. 2. Persistencia de costes: OPEX en SQLite local; para consultoria cloud habria que mover a Postgres. YAGNI por ahora (uso local). 3. Cobertura de tests del cliente: anadir React Testing Library / Vitest. 4. API key en servidor: OPENROUTER_API_KEY via entorno, no hardcodeado (correcto). 5. POD: el PDF se genera y guarda; falta servirlo como descarga en el frontend (ruta /pods/:file ya contemplada en api.js, pendiente de UI).

9. COMO VISITARLO COMO CONSULTORIA

1. Clonar: `git clone https://github.com/kavanasystemsinfo-ui/kavana-RouteFleet.git`
2. Backend: `cd server && npm install && npm test` (19 tests verdes)
3. Arrancar: `npm start` (puerto 5001)

4. Cliente: `cd client && npm install && npm run dev` (Vite, puerto 3001)
5. CI: ver pestaña Actions en GitHub -> workflow “CI - KAVANA RouteFleet”
6. Documentacion: README.md, DECISIONES_ESTRATEGICAS.md, task.md, server/README.md

El repositorio es publico y esta listo para revision tecnica.

Fin del informe de auditoria. Generado automaticamente tras sesion de profesionalizacion TDD + CI (2026-07-13).